DOCUMENT 14 — ERROR HANDLING, VALIDATION & EXCEPTION MANAGEMENT SPECIFICATION

1. Dokümanın Amacı

Bu doküman, Smart Chef & Pantry AI uygulamasında oluşabilecek hataların:

- Tespit edilmesini,
- Doğru katmanda ele alınmasını,
- Standardize edilmesini,
- Backend tarafından güvenli şekilde API response'una dönüştürülmesini,
- Flutter tarafından kullanıcıya anlaşılır şekilde gösterilmesini,
- Loglanmasını,
- Test edilebilmesini

tanımlar.

Bu dokümanın temel amacı, sistemde farklı modüllerin farklı hata formatları üretmesini engellemektir.

Tüm uygulamada ortak bir hata yönetimi yaklaşımı kullanılacaktır.

Temel prensip:

Hata oluşabilir.
Ancak hata davranışı kontrolsüz olmamalıdır.

---

2. Error Handling Kapsamı

Error Handling sistemi aşağıdaki katmanları kapsar:

Flutter UI
Flutter State
Use Case
Repository
Remote Data Source
Dio / Network
REST API
Flask Route

Schema Validation
Service
Repository
SQLAlchemy
PostgreSQL
Authentication
Authorization
Recommendation Engine
External Services

---

3. Temel Hata Yönetimi Prensibi

Sistemde:

Technical Error
↓
Exception
↓
Handled at appropriate layer
↓
Standard Error Model
↓
User-friendly UI message

akışı kullanılmalıdır.

Teknik hata doğrudan kullanıcıya gösterilmemelidir.

Örneğin PostgreSQL:

IntegrityError: duplicate key value violates unique constraint...

üretirse kullanıcıya bu mesaj gösterilmemelidir.

Bunun yerine:

Bu malzeme zaten kayıtlı.

gibi anlaşılır bir mesaj gösterilmelidir.

---

4. API Response Standardı

Document 05 ve Document 06 ile uyumlu olarak başarılı response:

```
{
  "success": true,
  "data": {},
  "message": "Operation completed successfully."
}
```

Hata response:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human readable message.",
    "details": {}
  }
}
```

şeklinde olacaktır.

---

5. Error Response Alanları

5.1 success

Her hata response'unda:

"success": false

olmalıdır.

5.2 code

Makine tarafından işlenebilecek sabit hata kodudur.

Örneğin:

AUTH_REQUIRED
INVALID_TOKEN
VALIDATION_ERROR
RECIPE_NOT_FOUND

Frontend bu kodlara göre davranabilir.

5.3 message

İnsan tarafından okunabilecek genel hata açıklamasıdır.

Örneğin:

"Recipe was not found."

5.4 details

İsteğe bağlı detay alanıdır.

Validation hatalarında:

```
{
  "field": "title"
}
```

veya:

```
{
  "fields": {
    "quantity": "Quantity must be greater than zero."
  }
}
```

gibi bilgiler bulunabilir.

Hassas bilgiler burada yer almamalıdır.

---

6. HTTP Status Code Kullanımı

Sistemde aşağıdaki HTTP status code'lar kullanılacaktır:

200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
500 Internal Server Error

---

7. 200 — OK

Başarılı GET, PATCH veya DELETE işlemlerinde kullanılabilir.

Örnek:

GET /api/cart

Response:

```
{
  "success": true,
  "data": {
    "items": []
  },
  "message": "Cart retrieved successfully."
}
```

---

8. 201 — Created

Yeni kaynak oluşturulduğunda kullanılmalıdır.

Örneğin:

POST /api/favorites

veya:

POST /api/admin/recipes

başarılı olduğunda:

201 Created

döndürülür.

---

9. 204 — No Content

Response body gerekmeyen başarılı işlemlerde kullanılabilir.

Ancak proje genelinde standart response yapısını korumak için:

```
{
  "success": true,
  "data": null,
  "message": "..."
}
```

kullanılması da tercih edilebilir.

MVP'de response standardizasyonu açısından body içeren standart response kullanmak daha tutarlı olacaktır.

---

10. 400 — Bad Request

İstek formatı veya parametreleri geçersiz olduğunda kullanılabilir.

Örneğin:

```
{
  "quantity": -3
}
```

gibi açıkça geçersiz bir değer gönderilmesi.

---

11. 401 — Unauthorized

Kullanıcı authentication gerektiren endpoint'e:

- JWT olmadan,
- geçersiz JWT ile,
- süresi dolmuş JWT ile

erişmeye çalışırsa kullanılır.

Örneğin:

GET /api/cart

Authorization header olmadan gönderilirse:

401 Unauthorized

---

12. 403 — Forbidden

Kullanıcı authenticated olduğu halde gerekli yetkiye sahip değilse kullanılır.

Örneğin:

Registered User
↓
POST /api/admin/recipes
↓
403 Forbidden

Error code:

ADMIN_REQUIRED

olabilir.

---

13. 404 — Not Found

İstenen kaynak bulunamadığında kullanılır.

Örnek:

RECIPE_NOT_FOUND
INGREDIENT_NOT_FOUND
FAVORITE_NOT_FOUND
CART_ITEM_NOT_FOUND

---

14. 409 — Conflict

İşlem mevcut veri ile çakıştığında kullanılır.

Örnek:

FAVORITE_ALREADY_EXISTS
INGREDIENT_ALREADY_EXISTS

---

15. 422 — Unprocessable Entity

Request yapısal olarak doğru olduğu halde business validation'dan geçemediğinde kullanılabilir.

Örneğin:

Recipe has no required ingredients.

Ancak 400 ve 422 kullanımının projede karışmaması için kesin kurallar belirlenmelidir.

Önerilen MVP yaklaşımı:

400 → malformed / invalid request
422 → semantically invalid input

---

16. 500 — Internal Server Error

Beklenmeyen backend hatalarında kullanılır.

Örneğin:

Unhandled Exception
Database Connection Failure
Unexpected Runtime Error

Ancak gerçek exception bilgisi kullanıcıya gösterilmemelidir.

Response:

```
{
  "success": false,
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "message": "An unexpected error occurred.",
    "details": {}
  }
}
```

---

17. Error Code Kategorileri

Hata kodları modüllere göre gruplanabilir.

Authentication

AUTH_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
SOCIAL_AUTH_FAILED
INVALID_PROVIDER

User

USER_NOT_FOUND

Validation

VALIDATION_ERROR
INVALID_UUID
INVALID_QUANTITY
EMPTY_INGREDIENT_LIST

Recipe

RECIPE_NOT_FOUND
RECIPE_INACTIVE
NO_RECIPES_FOUND

Ingredient

INGREDIENT_NOT_FOUND
INGREDIENT_ALREADY_EXISTS

Favorite

FAVORITE_NOT_FOUND
FAVORITE_ALREADY_EXISTS

Cart

CART_ITEM_NOT_FOUND
INVALID_CART_QUANTITY

Authorization

FORBIDDEN
ADMIN_REQUIRED

System

DATABASE_ERROR
INTERNAL_SERVER_ERROR

---

18. Backend Exception Hierarchy

Flask backend'de custom exception yapısı kullanılmalıdır.

Örneğin:

AppException
├── ValidationException

├── AuthenticationException
├── AuthorizationException
├── NotFoundException
├── ConflictException
└── DatabaseException

Bu exception'lar merkezi error handler tarafından HTTP response'a dönüştürülür.

---

19. Custom AppException

Temel exception yapısı:

AppException
- code
- message
- status_code
- details

Örneğin:

RecipeNotFoundException

şu değerleri taşıyabilir:

code = RECIPE_NOT_FOUND
status_code = 404
message = Recipe was not found.

---

20. Global Error Handler

Flask uygulamasında global error handler bulunmalıdır.

Akış:

Route
↓
Service
↓
Exception

↓
Global Error Handler
↓
Standard JSON

Örneğin:

RecipeService
↓
RecipeNotFoundException
↓
Flask Error Handler
↓
404 JSON Response

Bu sayede her route içerisinde ayrı ayrı hata response'u yazılması gerekmez.

---

21. Flask Error Handler Sorumluluğu

Global error handler:

- Exception türünü belirler.
- HTTP status belirler.
- Error code belirler.
- Standard JSON response üretir.
- Gerekiyorsa log oluşturur.
- Kullanıcıya teknik stack trace göndermez.

---

22. Development ve Production Hata Davranışı

Development ortamında:

FLASK_ENV=development

iken daha detaylı loglar tutulabilir.

Production'da:

FLASK_ENV=production

olduğunda:

- Stack trace API response'a gönderilmemeli.
- Database bilgileri response'a konulmamalı.
- Secret değerler loglanmamalı.
- Internal path bilgileri gösterilmemeli.

---

23. Validation Katmanları

Validation tek bir yerde yapılmamalıdır.

Üç seviyeli yaklaşım kullanılmalıdır:

1. Flutter Client Validation
2. API Schema Validation
3. Business Validation

---

24. Flutter Client Validation

Amaç kullanıcıya hızlı geri bildirim vermektir.

Örneğin Recipe Form:

Title boş

ise API'ye request göndermeden:

Tarif başlığı zorunludur.

gösterilebilir.

Ancak frontend validation güvenlik mekanizması değildir.

---

25. API Schema Validation

Backend'e gelen JSON request'in yapısını kontrol eder.

Örneğin:

```
{
  "quantity": "abc"
}
```

gibi yanlış tipte veri gönderilirse schema bunu reddeder.

Bu aşamada:

VALIDATION_ERROR

oluşturulabilir.

---

26. Business Validation

Schema valid olsa bile işlem business rule'a aykırı olabilir.

Örneğin:

```
{
  "quantity": 2
}
```

yapısal olarak doğrudur.

Ancak ingredient pasifse:

INGREDIENT_INACTIVE

veya uygun bir validation error döndürülmelidir.

Business validation Service katmanında yapılmalıdır.

---

27. Database Constraint

Son savunma katmanı database'dir.

Örneğin:

UNIQUE(user_id, recipe_id)

Favorite duplicate oluşmasını engeller.

Benzer şekilde:

CHECK(quantity > 0)

Cart quantity'nin geçersiz olmasını engelleyebilir.

Database constraint'leri application validation'ın yerine geçmez.

İkisi birlikte kullanılmalıdır.

---

28. Validation Flow

Genel akış:

Flutter
↓
Client Validation
↓
HTTP Request
↓
Schema Validation
↓
Authentication
↓
Authorization
↓
Business Validation
↓
Database Constraint
↓
Success

Her aşamada hata oluşabilir.

---

29. Authentication Error Handling

JWT yoksa:

AUTH_REQUIRED

JWT geçersizse:

INVALID_TOKEN

JWT süresi dolmuşsa:

TOKEN_EXPIRED

döndürülmelidir.

---

30. Flutter Token Expiration

Dio interceptor:

API Response
↓
401?
↓
TOKEN_EXPIRED
↓
Clear Token
↓
Auth State → Unauthenticated
↓
Login

akışını uygulayabilir.

Kullanıcıya:

Oturumunuz sona erdi. Lütfen tekrar giriş yapın.

gösterilebilir.

---

31. Unauthorized Request

Guest:

GET /api/favorites

gönderirse:

```
{
  "success": false,
  "error": {
    "code": "AUTH_REQUIRED",
    "message": "Authentication is required.",
    "details": {}
  }
}
```

Flutter bu durumda:

Login Page

yönlendirmesi yapabilir.

---

32. Authorization Error

Normal kullanıcı:

POST /api/admin/recipes

gönderirse:

```
{
  "success": false,
  "error": {
    "code": "ADMIN_REQUIRED",
    "message": "Admin access is required.",
    "details": {}
  }
```

}

Flutter bunu:

Bu işlem için yönetici yetkisi gereklidir.

şeklinde gösterebilir.

---

33. Recipe Error Handling

Recipe bulunamazsa:

RECIPE_NOT_FOUND

Pasif recipe doğrudan kullanıcı endpoint'inden istenirse:

RECIPE_INACTIVE

veya güvenlik/ürün tasarımına göre `RECIPE_NOT_FOUND` döndürülebilir.

MVP'de normal kullanıcıya pasif recipe detayının açılmaması tercih edilmelidir.

---

34. Ingredient Error Handling

Geçersiz ingredient:

INGREDIENT_NOT_FOUND

Duplicate ingredient:

INGREDIENT_ALREADY_EXISTS

Pasif ingredient'a yeni ilişki oluşturma:

VALIDATION_ERROR

veya ileride ayrı:

INGREDIENT_INACTIVE

kodu ile yönetilebilir.

---

35. Favorite Error Handling

Duplicate:

409
FAVORITE_ALREADY_EXISTS

Silinecek favorite bulunamıyorsa:

404
FAVORITE_NOT_FOUND

Unauthorized:

401
AUTH_REQUIRED

---

36. Cart Error Handling

Geçersiz quantity:

INVALID_CART_QUANTITY

CartItem bulunamazsa:

CART_ITEM_NOT_FOUND

Ingredient bulunamazsa:

INGREDIENT_NOT_FOUND

Unauthorized:

AUTH_REQUIRED

Ownership problemi:

FORBIDDEN

---

37. Recommendation Error Handling

Recommendation endpoint:

POST /api/recipes/recommend

boş ingredient listesi alırsa:

EMPTY_INGREDIENT_LIST

örneği:

```
{
  "success": false,
  "error": {
    "code": "EMPTY_INGREDIENT_LIST",
    "message": "At least one ingredient is required.",
    "details": {}
  }
}
```

---

38. Invalid UUID

Örneğin:

GET /api/recipes/abc

ve `abc` UUID değilse:

INVALID_UUID

döndürülmelidir.

Response:

```
{
  "success": false,
  "error": {
    "code": "INVALID_UUID",
    "message": "The provided ID is invalid.",
    "details": {
      "field": "recipe_id"
    }
  }
}
```

---

39. Database Error Handling

Database bağlantısı veya transaction sırasında hata oluşabilir.

Örneğin:

Database connection unavailable

Bu hata kullanıcıya:

DATABASE_ERROR

veya genel:

INTERNAL_SERVER_ERROR

olarak yansıtılabilir.

Gerçek SQL exception mesajı response'a konulmamalıdır.

---

40. Transaction Rollback

Database işlemi başarısız olursa transaction rollback edilmelidir.

Örneğin Recipe oluşturma:

BEGIN
↓
Create Recipe
↓
Create RecipeIngredient 1
↓
Create RecipeIngredient 2
↓
Error
↓
ROLLBACK

Sonuç:

Recipe oluşturulmamış olur.

Partial data bırakılmamalıdır.

---

41. Cart Transaction

Cart merge:

BEGIN
↓
Find existing item
↓
Update quantity
↓
COMMIT

Yeni item:

BEGIN
↓
Validate ingredient
↓
Create item
↓
COMMIT

hata halinde:

ROLLBACK

uygulanır.

---

42. Error Logging

Backend logları şu bilgileri içerebilir:

timestamp
request_id
endpoint
HTTP method
user_id (gerekliyse)
error_code
status_code
exception type

Ancak:

password
JWT
Google ID Token
client secret
database password

loglanmamalıdır.

---

43. Request ID

İleriye dönük olarak her API request'ine:

request_id

atanması önerilir.

Örneğin:

REQ-20260927-00123

gibi bir ID oluşturulabilir.

Log:

[REQ-20260927-00123]
POST /api/cart/items
ERROR INVALID_CART_QUANTITY

şeklinde tutulabilir.

Bu özellikle production debugging açısından faydalıdır.

MVP'de basit UUID request ID yeterlidir.

---

44. Flutter Exception Katmanı

Flutter'da:

core/errors/
├── api_exception.dart
├── failure.dart
└── exception_handler.dart

yapısı kullanılacaktır.

---

45. ApiException

API'den gelen hata için:

ApiException
- code
- message
- statusCode
- details

alanları bulunabilir.

Örneğin:

```
ApiException(
   code: "RECIPE_NOT_FOUND",
   message: "Recipe was not found.",
   statusCode: 404
)
```

---

46. Failure

Domain/UI katmanının doğrudan Dio veya HTTP detaylarını bilmemesi için `Failure` kullanılabilir.

Örneğin:

```
Failure
├── NetworkFailure
├── AuthenticationFailure
├── AuthorizationFailure
├── ValidationFailure
├── NotFoundFailure
└── UnknownFailure
```

Repository:

```
Exception
↓
Failure
```

dönüşümü yapabilir.

---

47. Dio Error Mapping

Dio tarafından alınan:

DioException

doğrudan UI'a gönderilmemelidir.

Akış:

DioException
↓
ExceptionHandler
↓
ApiException
↓
Failure
↓
Provider State
↓
UI

olmalıdır.

---

48. Network Error

Internet bağlantısı yoksa:

NetworkFailure

oluşabilir.

UI:

İnternet bağlantınızı kontrol edip tekrar deneyin.

mesajını gösterebilir.

---

49. Timeout

API request timeout olursa:

TimeoutFailure

veya `NetworkFailure` kullanılabilir.

UI:

İstek zaman aşımına uğradı. Lütfen tekrar deneyin.

gösterebilir.

---

50. Server Error

HTTP:

500

geldiğinde:

ServerFailure

oluşturulabilir.

UI:

Bir sunucu hatası oluştu. Lütfen daha sonra tekrar deneyin.

gösterebilir.

---

51. Error Message Mapping

Flutter'ın backend error code'larını kullanıcı mesajlarına çevirmesi önerilir.

Örneğin:

AUTH_REQUIRED
→ Giriş yapmanız gerekiyor.

TOKEN_EXPIRED
→ Oturumunuz sona erdi.

RECIPE_NOT_FOUND
→ Tarif bulunamadı.

INGREDIENT_NOT_FOUND
→ Malzeme bulunamadı.

FAVORITE_ALREADY_EXISTS
→ Bu tarif zaten favorilerinizde.

CART_ITEM_NOT_FOUND
→ Sepet ürünü bulunamadı.

INVALID_CART_QUANTITY
→ Geçerli bir miktar girin.

ADMIN_REQUIRED
→ Bu işlem için yönetici yetkisi gerekiyor.

NETWORK_ERROR
→ İnternet bağlantınızı kontrol edin.

---

52. UI Error Gösterim Kuralları

Küçük ve geçici hatalar:

SnackBar

ile gösterilebilir.

Örneğin:

Tarif favorilere eklenemedi.

Sayfanın yüklenememesi:

Full-page error state

ile gösterilebilir.

Form validation:

Field-level error

olarak gösterilmelidir.

---

53. Form Validation

Recipe Form:

Title
Description
Instructions
Category
Image URL
Source URL
YouTube URL
Ingredients

alanlarını kontrol etmelidir.

Örneğin:

Title boş

→

Tarif başlığı zorunludur.

---

54. Quantity Validation

Cart:

quantity > 0

Recipe ingredient:

quantity >= 0

veya quantity nullable ise:

quantity == null

kabul edilebilir.

Ancak negatif değer hiçbir durumda kabul edilmemelidir.

---

55. URL Validation

Kontrol edilmesi gereken alanlar:

image_url
source_url
youtube_url

URL:

- Geçerli formatta olmalı.
- Desteklenen scheme içermeli.
- `javascript:` gibi güvenli olmayan scheme'ler kabul edilmemelidir.

Önerilen scheme:

https

MVP'de mümkün olduğunca HTTPS kullanılmalıdır.

---

56. Text Validation

Title ve ingredient name için:

- Trim
- Boşluk kontrolü
- Minimum/maximum length
- Gereksiz whitespace temizliği

uygulanmalıdır.

Örneğin:

" Domates "

→

"Domates"

olmalıdır.

---

57. Pagination Validation

Pagination parametreleri:

page >= 1
limit >= 1
limit <= configured maximum

olmalıdır.

Örneğin:

?page=0&limit=10000

kabul edilmemelidir.

Önerilen maximum:

limit <= 100

olabilir.

---

58. Search Validation

Search query çok uzun veya kontrolsüz olmamalıdır.

Örneğin:

search.length <= configured maximum

kuralı uygulanabilir.

Search input database query'sine doğrudan string interpolation ile aktarılmamalıdır.

SQLAlchemy query parameterization kullanılmalıdır.

---

59. Validation Error Response

Birden fazla alan hatalıysa:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid.",
    "details": {
      "fields": {
        "title": "Title is required.",
        "youtube_url": "Invalid YouTube URL.",
        "ingredients": "At least one required ingredient is needed."
      }
    }
  }
}
```

kullanılabilir.

---

60. Flutter Validation Error Gösterimi

Örneğin:

Title
[_____]
Tarif başlığı zorunludur.

YouTube URL
[_____]
Geçerli bir YouTube bağlantısı giriniz.

şeklinde field-level gösterim yapılmalıdır.

---

61. Error State ve Retry

GET işlemlerinde hata oluştuğunda:

Error
[Tekrar Dene]

butonu bulunabilir.

Örneğin Favorites:

Favoriler yüklenemedi.

[Tekrar Dene]

Retry tekrar:

GET /api/favorites

çağrısı yapar.

---

62. Retry Kuralları

Her request otomatik olarak sonsuz kez tekrar edilmemelidir.

Retry yapılabilecek durumlar:

Temporary Network Failure
Timeout
5xx

Retry yapılmaması gereken durumlar:

400
401
403
404
409
422

Örneğin:

FAVORITE_ALREADY_EXISTS

hatasını tekrar tekrar göndermek çözüm değildir.

---

63. Authentication Retry

401 geldiğinde:

TOKEN_EXPIRED

ise auth flow tetiklenebilir.

Ancak aynı request'in sonsuz döngüye girmesi engellenmelidir.

Örneğin:

Request
↓
401
↓
Refresh/Login
↓
Retry once
↓
Still 401
↓
Logout

MVP'de refresh token kullanılmıyorsa doğrudan:

Clear JWT
→ Unauthenticated
→ Login

uygulanabilir.

---

64. Error Handling ve Navigation

Authentication hatası:

Protected Action
↓
401
↓
Login

Authorization:

Admin Action
↓
403
↓
Access Denied

404:

Recipe Detail
↓
Recipe Not Found
↓
Back / Home

şeklinde yönetilebilir.

---

65. Error Handling ve State Management

Riverpod state örneği:

AsyncValue.loading()
AsyncValue.data(data)
AsyncValue.error(error)

veya proje içerisinde özel state class'ları kullanılabilir.

Önemli olan:

Loading
Success
Empty
Error

durumlarının açıkça yönetilmesidir.

---

66. Error Handling'de Katman Sınırları

UI

Kullanıcıya gösterim.

Provider

State yönetimi.

UseCase

Domain operation.

Repository

Exception → Failure dönüşümü.

DataSource

HTTP/JSON iletişimi.

Backend Route

Request/response orchestration.

Schema

Input validation.

Service

Business validation.

Repository

Database access.

Database

Final constraints.

Her katmanın kendi sorumluluğu korunmalıdır.

---

67. Kesinlikle Yapılmaması Gerekenler

1. UI'dan Dio çağırmak

Page → Dio

olmamalıdır.

---

2. Route içerisinde database sorgusu

Route → SQLAlchemy

doğrudan yapılmamalıdır.

---

3. Frontend'e güvenmek

Flutter says user is admin

yeterli değildir.

---

4. Raw exception göstermek

IntegrityError(...)

kullanıcıya gösterilmemelidir.

---

5. Her hatayı 500 yapmak

Validation veya authorization hataları 500 olmamalıdır.

---

6. Her hatayı 200 döndürmek

Örneğin:

```
{
  "success": false
}
```

ile HTTP 200 kullanmak önerilmez.

HTTP status da gerçek sonucu ifade etmelidir.

---

68. Logging ve Monitoring Geleceği

MVP'de temel Flask logging yeterlidir.

İleride:

Sentry
OpenTelemetry
Prometheus
Grafana

gibi sistemler değerlendirilebilir.

Ancak MVP'de bunların kurulması zorunlu değildir.

---

69. Error Handling Test Matrisi

Durum	HTTP	Error Code
JWT yok	401	AUTH_REQUIRED
JWT geçersiz	401	INVALID_TOKEN
JWT expired	401	TOKEN_EXPIRED
Normal user admin endpoint	403	ADMIN_REQUIRED
Recipe yok	404	RECIPE_NOT_FOUND
Ingredient yok	404	INGREDIENT_NOT_FOUND
Favorite yok	404	FAVORITE_NOT_FOUND
Cart item yok	404	CART_ITEM_NOT_FOUND

Duplicate favorite	409	FAVORITE_ALREADY_EXISTS
Duplicate ingredient	409	INGREDIENT_ALREADY_EXISTS
Geçersiz quantity	400/422	INVALID_CART_QUANTITY
Boş ingredient listesi	400/422	EMPTY_INGREDIENT_LIST
Invalid UUID	400	INVALID_UUID
Database error	500	DATABASE_ERROR
Beklenmeyen hata	500	INTERNAL_SERVER_ERROR

---

70. Modül Bazlı Hata Akışı

Auth

Google Token
↓
Validation
↓
Social Auth
↓
JWT

Hata:

SOCIAL_AUTH_FAILED

Recipe

Recipe Request
↓
Validation
↓
RecipeService
↓
Repository

Hata:

RECIPE_NOT_FOUND
VALIDATION_ERROR
DATABASE_ERROR

Ingredient

Ingredient Request
↓
Validation
↓
IngredientService
↓
Repository

Hata:

INGREDIENT_NOT_FOUND
INGREDIENT_ALREADY_EXISTS

Favorites

Favorite Request
↓
JWT
↓
Ownership
↓
FavoriteService
↓
Repository

Hata:

FAVORITE_ALREADY_EXISTS
FAVORITE_NOT_FOUND

Cart

Cart Request
↓

JWT
↓
Validation
↓
CartService
↓
Merge / Update
↓
Repository

Hata:

INVALID_CART_QUANTITY
CART_ITEM_NOT_FOUND

---

71. End-to-End Error Example

Senaryo:

Kullanıcı Recipe Detail'den:

Eksik Malzemeleri Sepete Ekle

butonuna basıyor.

JWT süresi dolmuş.

Akış:

Flutter
↓
POST /api/cart/items
↓
Flask JWT
↓
TOKEN_EXPIRED
↓
401
↓
Dio Interceptor
↓
Clear JWT
↓

Auth State = Unauthenticated
↓
Login Screen

Kullanıcıya:

Oturumunuz sona erdi. Lütfen tekrar giriş yapın.

gösterilir.

---

72. End-to-End Validation Example

Admin:

Recipe Create

yapar.

Malzeme listesinde:

ingredient_id = invalid UUID

bulunur.

Akış:

Flutter Validation
↓
API
↓
Schema
↓
INVALID_UUID
↓
400
↓
Flutter
↓
Form Error

Database'e hiçbir kayıt yazılmamalıdır.

---

73. End-to-End Database Error Example

Admin recipe oluşturur.

Recipe oluşturulur ancak RecipeIngredient kayıtlarından biri database constraint'e takılır.

Beklenen:

BEGIN
↓
Create Recipe
↓
Create Ingredients
↓
Database Error
↓
ROLLBACK
↓
DATABASE_ERROR

Database'de yarım recipe kalmamalıdır.

---

74. Final Error Architecture

FLUTTER
_____

UI
↓
Provider
↓
UseCase
↓
Repository
↓
Exception Mapper
↓
Failure
↓
UI Message

HTTP / REST
──

Flask Route
↓
Schema Validation
↓
Authentication
↓
Authorization
↓
Service
↓
Repository
↓
SQLAlchemy
↓
PostgreSQL
↓
Exception
↓
Global Error Handler
↓
Standard Error Response
↓
Flutter Exception Mapper

---

75. Final Error Response Standardı

Başarılı işlem:

```
{
  "success": true,
  "data": {},
  "message": "Operation completed successfully."
}
```

Başarısız işlem:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human readable message.",
    "details": {}
  }
}
```

Bu iki yapı uygulamanın tamamında mümkün olduğunca korunmalıdır.

---

76. MVP Error Handling Kuralları

1. Tüm API hataları standardize edilmelidir.
2. Her hata uygun HTTP status code ile dönmelidir.
3. Teknik exception kullanıcıya gösterilmemelidir.
4. Authentication ve authorization birbirinden ayrılmalıdır.
5. Validation üç katmanda ele alınmalıdır.
6. Frontend validation güvenlik mekanizması değildir.
7. Business validation Service katmanında yapılmalıdır.
8. Database constraint'leri korunmalıdır.
9. Transaction başarısız olursa rollback yapılmalıdır.
10. Global Flask error handler kullanılmalıdır.
11. Flutter'da Dio exception doğrudan UI'a aktarılmamalıdır.
12. API error code'ları Flutter tarafından map edilmelidir.
13. Kullanıcıya anlaşılır mesajlar gösterilmelidir.
14. Hassas bilgiler loglanmamalıdır.
15. Retry yalnızca uygun hata türlerinde uygulanmalıdır.
16. 401 durumunda authentication state güncellenmelidir.
17. 403 durumunda kullanıcı yetkisiz olduğu konusunda bilgilendirilmelidir.
18. 404 durumunda ilgili kaynak için uygun empty/error state gösterilmelidir.
19. 409 duplicate durumlarında kullanıcıya anlamlı bilgi verilmelidir.
20. 500 hatalarında teknik detaylar gizlenmelidir.

---

77. Document 14 Sonuç

Smart Chef & Pantry AI'de Error Handling yalnızca hata mesajı göstermekten ibaret değildir.

Sistemin:

Validation
+
Authentication
+
Authorization
+
Business Rules
+
Database Constraints
+
Transactions
+
Exception Mapping
+
User Feedback

katmanlarının birlikte çalışmasını sağlayan merkezi bir mimari oluşturur.

Son hedef:

Technical Exception
↓
Controlled Backend Exception
↓
Standard API Error
↓
Flutter Failure
↓
User-friendly Message

akışını bütün uygulamada standart hale getirmektir.

Bu yapı sayesinde:

- Recipe işlemleri,
- Ingredient işlemleri,
- Recommendation,
- Favorites,
- Cart,
- Authentication,
- Admin işlemleri

aynı hata yönetim standardını kullanacaktır.

Böylece proje büyüdüğünde her yeni feature için ayrı bir hata sistemi oluşturulmasına gerek kalmayacak ve uygulamanın backend ile Flutter tarafındaki davranışı daha öngörülebilir, test edilebilir ve sürdürülebilir olacaktır.